“Warehouse Inventory Management and Optimization System”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms - 3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

PUTTI MOKSHITHA	**:2520030396**
ABHISHIKTHA	**:2520030185**

Team Number: 17

Section: S-6

Under the Guidance of

Dr. V. Sireesha

Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Warehouse Inventory Management and Optimization System"** is a record of Bonafide work of **Putti Mokshitha – 2520030396, Abhishiktha – 25200303185** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Putti Mokshitha – 2520030396

Abhishiktha – 25200303185

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled **"Warehouse Inventory Management and Optimization System"** is a bonafide work done and submitted by **PUTTI MOKSHITHA – 2520030396, ABHISHIKTHA – 25200303185** in partial fulfillment of the

requirements for the award of the degree of**BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide **Signature of the HOD**
(Dr. V. Sireesha)

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. V. Sireesha, Professor, Department of Computer Science and Engineering, for the guidance, technical suggestions, encouragement, and continuous support provided throughout the development and documentation of this project. The feedback received helped us connect theoretical algorithmic concepts with practical warehouse-management requirements.

We also thank Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the academic environment and facilities required to complete the project. We are grateful to the faculty members and laboratory staff who supported the learning, implementation, testing, and evaluation process.

Finally, we thank our teammates, classmates, and everyone who contributed directly or indirectly to the completion of the Warehouse Inventory Management and Optimization System. The project provided practical experience in algorithm design, Java implementation, data processing, testing, debugging, performance analysis, and technical documentation.

Putti Mokshitha – 2520030396

Abhishiktha – 25200303185

Abstract

Managing inventory across multiple warehouses requires fast search, accurate restocking, and efficient stock transfers. This project presents a Warehouse Inventory Management and Optimization System designed to keep the right stock in the right place. Product records for all warehouses are maintained in shared files and accessed through both a browser dashboard and a terminal interface.

The system enables:

- Product search by name or SKU with automatic typo correction.
- Budget-aware restocking plans that maximize benefit within fixed limits.
- Stock transfer optimization across warehouses, respecting route capacities.
- Risk scoring for stockouts and supplier selection to cover at-risk products.
- Cycle counts supported by fair random sampling.
- Parallel data analysis for datasets up to ten million rows.

All core algorithms are implemented from scratch in Java and verified by automated tests. These include KMP, Aho-Corasick, edit distance, knapsack dynamic programming, maximum flow, greedy set cover, quickselect, and reservoir sampling. Together, they deliver a scalable, reliable, and intelligent solution that enhances warehouse productivity and ensures robust inventory control.

Table of Contents

Chapter	Contents	Page No.
Acknowledgement		
Abstract		
Table of Contents		

List of Figures

1	Introduction	1.1 Overview 1.2 Background and Motivation 1.3 Problem Statement 1.4 Scope of the Project 1.5 Objectives 1.6 Challenges Addressed 1.7 Course Syllabus Topics Applied 1.8 Chapter Organization	1–6
2	System Requirements	2.1 Related Work and Algorithmic Context 2.2 Functional Requirements 2.3 Non-Functional Requirements 2.4 Hardware Requirements 2.5 Software Requirements 2.6 Dataset Requirements 2.7 User Requirements 2.8 Summary of Requirements	7–12
3	System Design	3.1 Design Goals 3.2 High-Level Architecture 3.3 Layer Description 3.4 Data Flow 3.5 Module Mapping 3.6 Inventory Status and Risk Model 3.7 Data Structures Used 3.8 Algorithm-to-Feature Mapping 3.9 Complexity Analysis 3.10 Datasets Used 3.11 Tools and Technologies 3.12 Suitability of the Chosen Approach	13–21
4	Implementation	4.1 Implementation Overview 4.2 Authentication and Access Control 4.3 Inventory and CSV Processing 4.4 Intelligent Search 4.5 String Laboratory 4.6 Replenishment 4.7 Warehouse Balancing 4.8 Risk and Supplier Prioritization 4.9 Cycle Counting 4.10 Large Data Processing	22–36

		4.11 API Integration 4.12 Front-End Workflow 4.13 Summary of Implementation	
5	Results	5.1 Test Environment 5.2 Functional Test Strategy 5.3 Individual Test Results 5.4 Performance Results 5.5 Dashboard Results 5.6 Replenishment Results 5.7 Warehouse Balancing Results 5.8 Search Results 5.9 Large-Data Results 5.10 Complexity and Interpretation 5.11 Project Screenshots	37–47
6	Challenges & Limitations	6.1 Issues Faced During Development 6.2 Solutions Applied 6.3 Limitations of the System 6.4 Security and Operational Considerations 6.5 Summary of Challenges and Limitations	48–51
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills Learned During Development	52–54
	References	Books, journal papers, and documentation sites used	55–56

List of Figures

1.1	From inventory records to operational warehouse decisions	2
3.1	High-level architecture showing the interfaces, Java application layer, algorithm engine, CSV persistence, and result files	14
3.2	Integrated workflow connecting inventory analysis with search, replenishment, warehouse balancing, risk prioritization, and cycle counting	16
4.1	Intelligent search flow combining tokenization, Aho-Corasick matching, ranking, and Damerau-distance typo correction	25

4.2	Java source implementation of the intelligent search index and tokenization logic	26
4.3	Java source implementation showing the hand-built algorithm engine, pseudo-random generator, and KMP string-matching logic	28
4.4	Replenishment flow from low-stock detection to knapsack optimization, preview, confirmation, and CSV update	30
5.1	Reported execution times for selected large-dataset operations	40
5.2	System overview dashboard showing warehouse-wide operational status	43
5.3	Inventory management interface showing product records and stock status	43
5.4	Restocking planner showing the budget-constrained replenishment plan	44
5.5	Warehouse transfer planner showing maximum-flow based stock movements	44
5.6	Alerts and priority view showing risk-ranked products	45
5.7	Cycle-count planner showing reservoir-sampled products for physical verification	45
5.8	Data center interface for large-dataset generation, analysis, sampling, and search	46
5.9	Warehouse Control Center login interface used for authenticated access	46
5.10	Terminal interface showing the authenticated warehouse system and available inventory-management operations	47

1. INTRODUCTION

1.1 Overview

Warehouse inventory management is the process of recording, monitoring, controlling, and optimizing goods across storage locations. In a multi-warehouse environment, the same product may be sufficiently stocked at one location while being below its required level at another. Therefore, inventory management is not only a data-entry task. It is a decision problem involving search, prioritization, allocation, optimization, and uncertainty.

The Warehouse Inventory Management and Optimization System was developed as an integrated Java application that connects routine warehouse operations with algorithmic techniques from Data Structures and Algorithms. The system provides a browser-based dashboard as well as a terminal interface. Both interfaces work with the same underlying inventory store and business logic.

The project contains product records with fields such as identifier, SKU, product name, category, warehouse, supplier, quantity, minimum stock, unit price, and demand. The supplied data includes five warehouse locations: Hyderabad, Mumbai, Delhi, Chennai, and Kolkata. Route information is also maintained for stock-transfer planning.

The important design principle is that each algorithm is connected to a concrete operational task. Search algorithms such as Knuth-Morris-Pratt (KMP) [1] and Aho-Corasick [2] are used to locate products even when queries contain multiple terms, and Damerau edit distance [3] tolerates typing errors. Dynamic programming, in the form of the 0/1 knapsack formulation [4], is used to select replenishment candidates under a budget. Maximum flow, computed with the Edmonds-Karp algorithm [5], is used to determine feasible stock transfers. Greedy set cover [6] is used to prioritize suppliers. Reservoir sampling [7] supports cycle counting, while streaming, quickselect [8], and parallel processing support large inventory

datasets. The whole system is implemented in Java [9] without an external database or web framework.

The datasets, test results, and performance measurements discussed in this report are taken from the project source code and the supplied project documentation [10]. Fig. 1.1 summarizes how raw inventory records are converted into operational warehouse decisions.

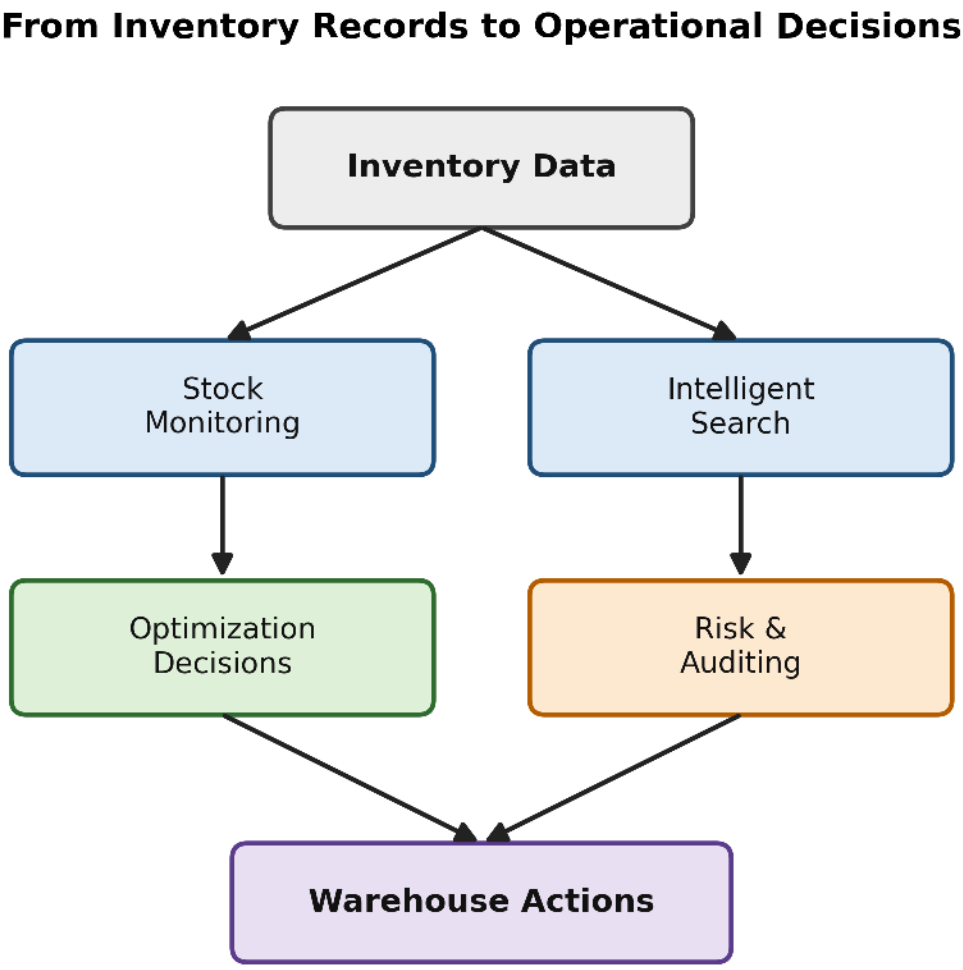


Fig. 1.1 From inventory records to operational warehouse decisions

1.2 Background and Motivation

Traditional inventory software often focuses on CRUD operations: creating, reading, updating, and deleting records. Those operations are necessary, but they do not by themselves answer the decisions that warehouse administrators face every day.

A low-stock report, for example, can identify products below their minimum levels, but it does not automatically determine which subset should be purchased when the purchasing budget is limited. Similarly, a list of warehouses with shortages does not determine whether available surplus can actually reach those warehouses when transport routes have capacity constraints.

The project therefore uses the warehouse domain as a practical setting for demonstrating multiple DSA concepts. This makes the application useful both as an academic DSA demonstration and as a small decision-support prototype.

1.3 Problem Statement

The project addresses the problem of designing a single warehouse inventory platform in which routine inventory management is combined with algorithmic decision support. The system must provide reliable product search, identify inventory risk, generate replenishment plans under a budget, balance stock across warehouses subject to route capacities, prioritize suppliers that cover risky products, support cycle counting, and process much larger datasets without requiring the entire dataset to be loaded into memory.

The problem is challenging because each operation has a different computational structure. Search is a string-processing problem; replenishment is an optimization problem; warehouse balancing is a network-flow problem; supplier prioritization is an approximation problem; cycle counting is a randomized streaming problem; and large-data analysis is a systems and data-processing problem.

1.4 Scope of the Project

- Inventory creation, update, deletion, search, and dashboard reporting.
- Warehouse- and category-level summaries and inventory valuation.
- Risk scoring based on stock coverage, minimum-stock

requirements, and out-of-stock conditions.

- Budget-constrained replenishment using 0/1 knapsack dynamic programming.
- Warehouse balancing using maximum-flow modeling and Edmonds-Karp.
- Supplier prioritization using a greedy set-cover approximation.
- Cycle-count sampling and optional inventory adjustment.
- String-algorithm experimentation using KMP, Z-function, Rabin-Karp, Aho-Corasick, suffix arrays, and LCP computation.
- Large-dataset generation, parallel aggregation, streaming search, reservoir sampling, and percentile analysis.
- Browser and terminal interfaces using common CSV-based persistence.

1.5 Objectives

1. Develop a practical Java warehouse inventory management application.
2. Integrate classical data structures and algorithms into realistic warehouse workflows.
3. Provide intelligent product search with multi-pattern matching and typo correction.
4. Optimize replenishment decisions under a limited purchasing budget.
5. Balance inventory across warehouses using network-flow techniques.
6. Prioritize risky inventory and suppliers using rule-based scoring and approximation.
7. Support scalable analysis of large inventory datasets using streaming and parallel techniques.
8. Validate algorithmic correctness and application behavior using automated tests.

9. Document performance behavior on large datasets.
10. Provide a usable browser and terminal interface over a shared inventory store.

1.6 Challenges Addressed

- Converting warehouse business rules into precise computational definitions.
- Keeping browser and terminal operations consistent.
- Turning optimization results into understandable operational plans.
- Handling residual networks and route capacities correctly.
- Maintaining agreement between several independent string-search implementations.
- Processing very large files without loading the entire file into memory.
- Updating a shared CSV safely.
- Providing typo-tolerant search without making exact matches unreliable.

1.7 Course Syllabus Topics Applied

This project was carried out for the course Data Structures and Algorithms - 3 (25CS2103E), whose syllabus is organized into six modules. The central idea of Module 1 is that every practical query or task can be mapped to an advanced-algorithm family: pattern search maps to string algorithms, fuzzy matching to dynamic programming, flow problems to maximum flow, scheduling and coverage problems to approximation, and sampling problems to randomised algorithms. The warehouse system follows this idea directly, and in line with the course requirement that algorithms be hand-built, the algorithms in the project are implemented from scratch in Java. Table 1.1 lists the syllabus topics from each module that were used in the project and where they appear.

Table 1.1 Course modules and the topics applied in the project

Module	Syllabus Topics Used	Application in the Project
Module 2: String Algorithms	Knuth-Morris-Pratt and the failure function; Z-function; Rabin-Karp with rolling hash; Aho-Corasick multi-pattern matching; suffix arrays; LCP array.	Intelligent product search, the String Laboratory, and streaming search over large files (Sections 4.4 and 4.5).
Module 3: Advanced DP	Edit distance and its variants (Damerau-Levenshtein); the DP framework with overlapping subproblems and optimal substructure; knapsack-style DP; recognizing where greedy suffices.	Typo correction in search using Damerau edit distance, and budget-constrained replenishment using 0/1 knapsack DP (Sections 4.4 and 4.6).
Module 4: Network Flow	The max-flow problem with source, sink, capacity and conservation; residual graph and the Ford-Fulkerson method; Edmonds-Karp with BFS augmenting paths; max-flow min-cut consistency.	Warehouse balancing, where surplus warehouses feed shortage warehouses through capacity-limited routes (Section 4.7).
Module 5: NP-Completeness & Approximation	Recognizing NP-hard structures such as set cover; approximation algorithms and ratio guarantees; greedy approximation.	Supplier prioritization using the greedy set-cover approximation (Section 4.8).
Module 6: Randomised & Parallel Algorithms	Randomised hashing; randomised selection (quickselect); reservoir sampling for streaming data; parallel reduce and data-parallel processing.	Rabin-Karp hashing, percentile analysis with quickselect, cycle-count sampling, and parallel aggregation of large files (Sections 4.9 and 4.10).

Syllabus topics that are not listed in Table 1.1, such as suffix automata and suffix trees, Dinic's algorithm, bipartite matching, and primality testing, were not required by the warehouse problem and are outside the

scope of this project. The syllabus textbooks includeCormen et al. [4], which is also used as a reference in this report.

1.8 Chapter Organization

Chapter 2 defines system requirements, algorithmic context, functional and non-functional requirements, hardware and software needs, data requirements, and the administrator user. Chapter 3 describes the architecture, data flow, module mapping, inventory risk model, data structures, algorithms, tools, and complexity. Chapter 4 explains the implementation module by module and provides pseudocode for the principal algorithms. Chapter 5 presents tests, results, performance observations, output screenshots, and interpretation. Chapter 6 records development challenges, solutions, limitations, and operational security considerations. Chapter 7 concludes the report and presents future improvements and skills learned.

2. SYSTEM REQUIREMENTS

2.1 Related Work and Algorithmic Context

The system combines established algorithmic techniques with an inventory-management application. Knuth-Morris-Pratt provides efficient exact pattern matching without repeatedly rechecking characters [1]. Aho-Corasick extends trie-based matching to multiple patterns [2]. Rabin-Karp uses rolling hashes [11], while the Z-function provides another linear-time string-processing technique [12]. Suffix arrays and LCP information provide a compact structure for suffix-based analysis [13].

Approximate matching is motivated by edit-distance techniques. Damerau edit distance extends insertion, deletion, and substitution by accounting for adjacent transposition, which is useful for common typing errors in product queries [3].

The replenishment module is related to the 0/1 knapsack problem: each candidate order has a cost and an associated benefit, and the total

selected cost must remain within a budget [4]. Warehouse balancing is naturally represented as a capacitated flow network solved with the Edmonds-Karp algorithm [5]. Supplier prioritization resembles set cover, where each supplier covers a subset of risky products [6]. Cycle counting is suitable for reservoir sampling because the sample can be maintained in one pass over a stream [7]. Percentile analysis on large numeric data can use the selection idea introduced by Hoare [8].

The project does not claim that these algorithms solve every real warehouse problem optimally. Instead, they provide formal, explainable approaches for the academic prototype and demonstrate how algorithm selection depends on the structure of the task.

2.1.1 Gaps and Limitations in Existing Work

Although each of the above techniques is well studied, the existing literature and standard textbook implementations present them in isolation, usually on abstract inputs such as arrays of integers, random strings, or textbook graphs. They rarely show how several techniques can be combined in one working application that operates on realistic business data.

Conventional inventory software, on the other hand, concentrates on record keeping and reporting. A low-stock report does not choose which purchases to make within a limited budget, and a shortage list does not check whether transport routes can physically carry the surplus stock. Typo-tolerant search, supplier coverage analysis, and statistically fair audit sampling are also usually missing from simple inventory tools. Finally, many small academic implementations load whole files into memory and therefore do not scale to datasets of millions of rows.

This project attempts to fill these gaps by (i) mapping each algorithm to one concrete warehouse task, (ii) exposing the results through a common browser and terminal interface, (iii) validating each algorithm against reference behavior, and (iv) using streaming and parallel processing so that large datasets can be handled. The limitations that remain are

discussed in Chapter 6.

2.2 Functional Requirements

The functional requirements describe the functions that the system must perform. They are summarized in Table 2.1 and elaborated below.

Table 2.1 Functional requirements of the system

ID	Requirement	Description
FR1	Authentication	Allow users to log in and maintain a session.
FR2	Admin Access Control	Restrict all system operations to the authenticated administrator.
FR3	Product Management	Add, update, delete, and view inventory records by the authenticated administrator.
FR4	Search	Search products by SKU, name, and multiple query terms.
FR5	Typo Correction	Suggest likely vocabulary terms when a query is misspelled.
FR6	Dashboard	Display product count, units, inventory value, low-stock, out-of-stock, surplus, warehouse, and category summaries.
FR7	Replenishment	Generate and apply a budget-constrained replenishment plan.
FR8	Balancing	Generate and apply warehouse-to-warehouse transfers subject to route capacities.
FR9	Risk and Priority	Rank risky products and prioritize suppliers covering at-risk products.
FR10	Cycle Counting	Generate a random audit sample and record discrepancies.
FR11	String Laboratory	Run and compare multiple string-search algorithms.
FR12	Large Data Center	Generate, aggregate, search, sample, and compute percentiles on large datasets.
FR13	Results Export	Store operational and analytical outputs as CSV result files.

2.2.1 Authentication

The application shall validate credentials against the supplied user data, create a session token after successful authentication, and reject invalid credentials.

2.2.2 Inventory CRUD

The application shall support inventory creation and modification for the authenticated administrator only.

2.2.3 Search and Ranking

The search workflow shall support exact SKU matches, multiple terms, phrase scoring, and typo suggestions.

2.2.4 Optimization Preview

Replenishment and balancing shall expose a preview/apply distinction so an operator can inspect a proposed change before applying it.

2.2.5 Large-Data Processing

The large-data functionality shall support streaming and parallel operations without requiring the complete large file to be held in memory.

2.3 Non-Functional Requirements

The non-functional requirements describe the quality attributes expected from the system, such as response time, usability, scalability, and reliability. They are listed in Table 2.2.

Table 2.2 Non-functional requirements of the system

Attribute	Requirement	Project Interpretation
Performance	Interactive operations should return promptly.	Normal inventory operations use in-memory structures and direct file persistence.
Scalability	Large files should not require complete memory loading.	Streaming and byte-range processing support the documented 10-million-record experiments.
Usability	Functions should be	Browser dashboard and

Reliability	accessible through clear workflows.	terminal menu expose the same core operations.
Security	Authentication and authorization are required.	Salted password hashes, random session tokens, and backend admin-session checks are used.
Maintainability	Algorithmic logic should be separable from UI logic.	Storage, algorithms, API handlers, and presentation are conceptually separated.
Data integrity	Updates should not leave partial files.	Temporary-file replacement is used for inventory writes.

2.4 Hardware Requirements

- Modern multi-core CPU is recommended for parallel large-data processing.
- At least 4 GB RAM is recommended for normal operation; more memory is beneficial for large experiments.
- Sufficient storage is required for application files, result files, and the documented large dataset of approximately 950 MB.
- Standard keyboard, mouse, and browser-capable display are sufficient for the web interface.

2.5 Software Requirements

- Java Development Kit with javac and Java runtime.
- Operating system capable of running Java and the supplied shell/ batch launcher.
- Modern web browser.
- No external database server is required because CSV files provide persistence.
- Supplied run.sh or run.bat launcher.

2.6 Dataset Requirements

The core inventory dataset contains id, sku, name, category, warehouse,

supplier, qty, min, price, and demand. The route dataset contains from, to, and capacity_units fields. A users dataset stores the administrator's authentication information. The project also supports generation of a large inventory dataset for scalability experiments.

The large-data experiment is explicitly treated as a scalability benchmark. The normal application inventory and the generated large dataset serve different purposes: the normal inventory supports interactive operations, while the large dataset demonstrates streaming, parallel aggregation, search, sampling, and percentile processing.

2.7 User Requirements

The system has a single user type, the Admin, who has full access to all functions. No other user roles are defined. The Admin user is described in Table 2.3.

Table 2.3 Admin user and permitted operations

User	Permitted Operations
Admin	Log in; view the dashboard; add, update, and delete inventory records; search products; analyze risk; preview and apply replenishment and balancing plans; perform cycle counts; use the string laboratory and the large data center.

2.8 Summary of Requirements

The requirements above define a system that is functionally broad but technically lightweight: it needs only a Java runtime, a browser, and CSV files. The non-functional requirements place particular emphasis on correctness, scalability for large files, and data integrity, and these three concerns drive many of the design decisions explained in Chapter 3.

3. SYSTEM DESIGN

3.1 Design Goals

The system design is based on five goals: correctness, algorithmic

transparency, operational usability, scalability, and data consistency. Correctness means that algorithm outputs are tested against reference behavior. Algorithmic transparency means that each important warehouse function has an identifiable DSA technique. Operational usability means that the user sees warehouse decisions rather than raw algorithm internals. Scalability means that large files can be processed without materializing the entire dataset. Data consistency means that the browser and terminal interfaces work with the same underlying inventory state.

3.2 High-Level Architecture

The application is organized into presentation, application/API, algorithm and business-logic, and persistence concerns. The browser interface sends HTTP requests to the Java server. The terminal interface invokes the same underlying capabilities. Inventory data is stored in CSV files, and analytical plans are written to result files. The overall architecture is shown in Fig. 3.1.

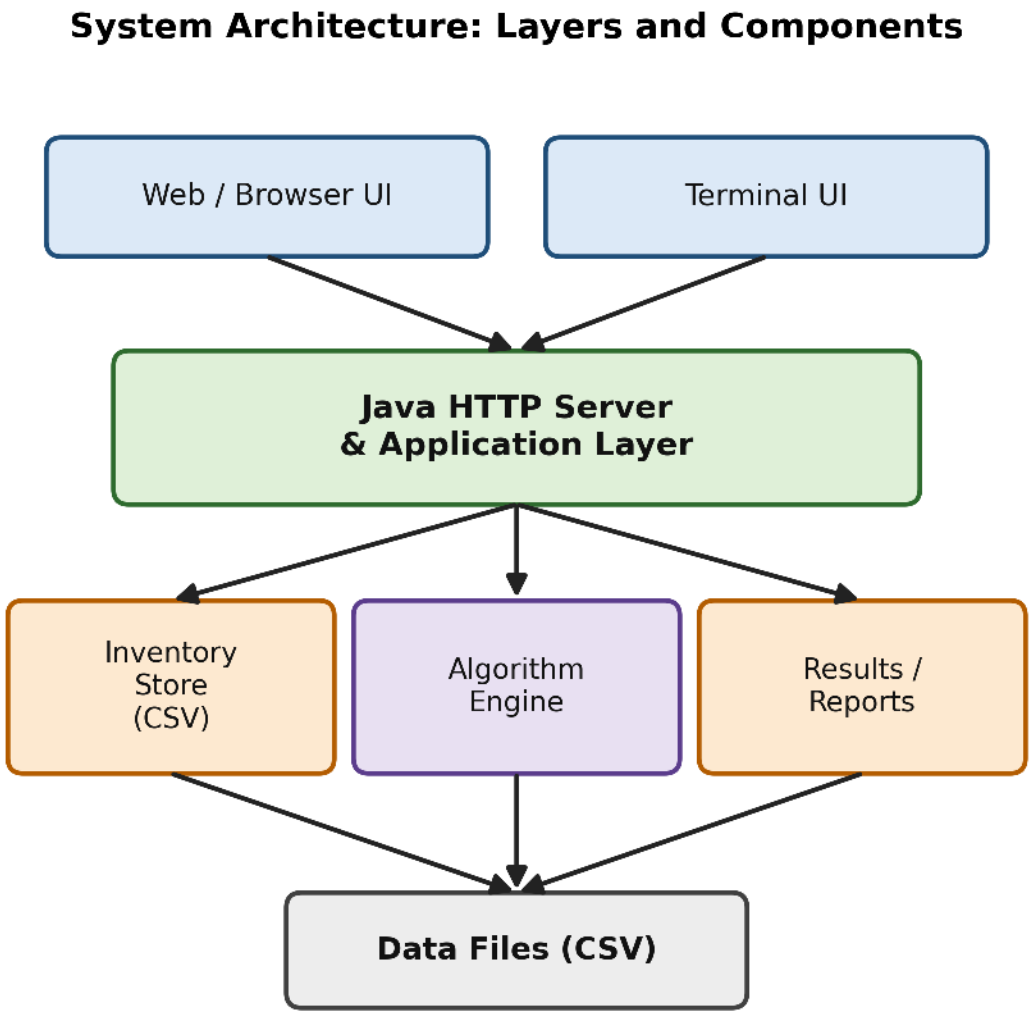


Fig. 3.1 High-level architecture showing the interfaces, Java application layer, algorithm engine, CSV persistence, and result files

3.3 Layer Description

3.3.1 Presentation Layer

The browser page provides the operational dashboard, product workflows, search, optimization controls, and result displays. The terminal interface provides an alternative interaction path over the same project logic.

3.3.2 API and Application Layer

The Java HTTP server receives requests, validates the admin session, invokes business operations, and serializes results for the browser.

3.3.3 Algorithm Layer

The algorithm engine contains search, optimization, graph, greedy, randomized, and large-data processing logic.

3.3.4 Persistence Layer

CSV files provide transparent persistence for inventory, routes, and users. Result files preserve plans and analytical outputs.

3.4 Data Flow

A typical request begins when the user enters an operation in the browser or terminal. The request is validated, the relevant inventory state is loaded or refreshed, the algorithmic module is executed, and the result is returned for display. For operations that change data, the project uses a preview/apply separation where applicable. The final update is persisted to CSV using a safe replacement process. Fig. 3.2 shows how inventory analysis connects to the search, replenishment, balancing, risk, and cycle-counting workflows.

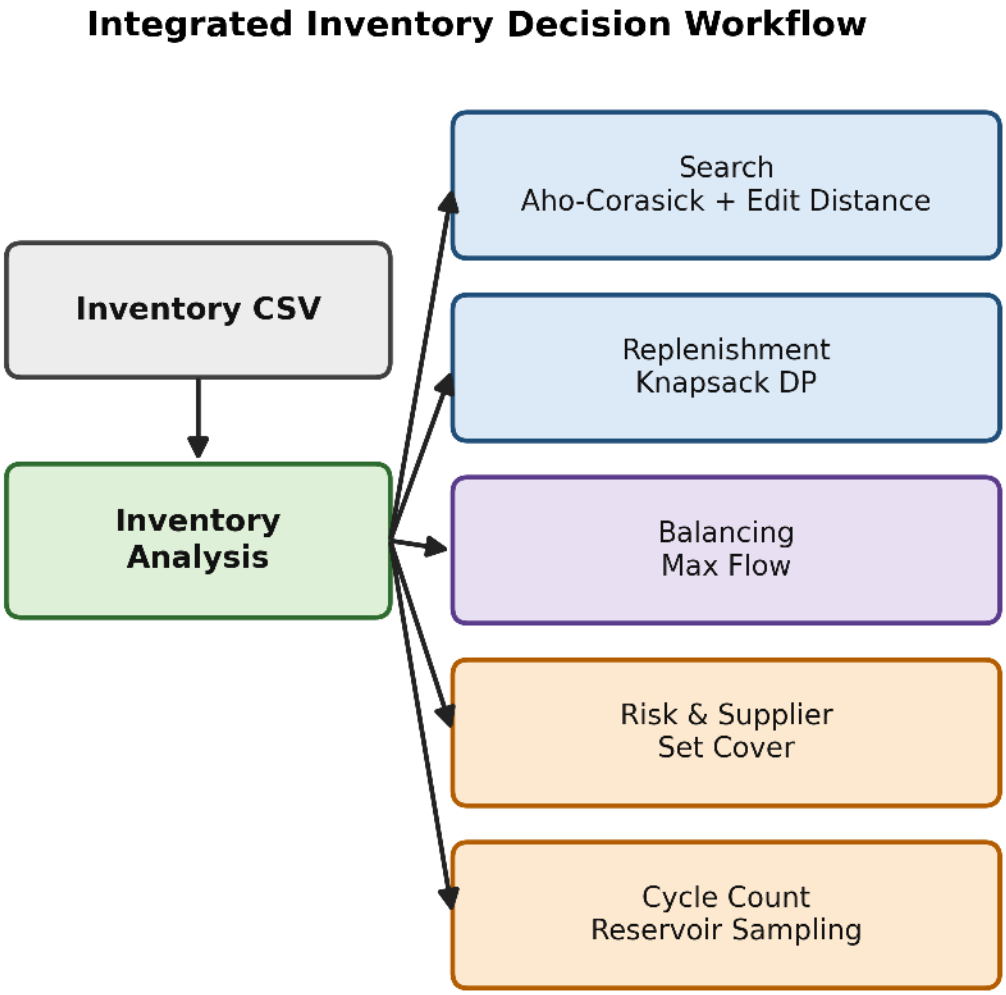


Fig. 3.2 Integrated workflow connecting inventory analysis with search, replenishment, warehouse balancing, risk prioritization, and cycle counting

3.5 Module Mapping

The system is divided into six modules. Table 3.1 maps each module to its warehouse feature and the main data structure or algorithm used.

Table 3.1 Module mapping of the system

Module	Warehouse Feature	Primary DSA / Technique	Purpose
M1	Inventory Search	Aho-Corasick, KMP, Z-function, Rabin-Karp	Efficient product search and string analysis.
M2	Typo Correction	Damerau edit distance	Handle spelling

			and transposition errors.
M3	Replenishment	0/1 Knapsack DP	Select useful restocks within a budget.
M4	Warehouse Balancing	Edmonds-Karp maximum flow	Move stock subject to route capacities.
M5	Risk & Priority	Rule-based scoring + Greedy Set Cover	Rank risk and prioritize suppliers.
M6	Cycle Counting / Large Data	Reservoir sampling + streaming + parallel aggregation + quickselect	Audit sampling and scalable analysis.

3.6 Inventory Status and Risk Model

The inventory status model classifies products using the relationship between current quantity and minimum stock. An out-of-stock item has quantity at or below zero. A low-stock item has quantity below the minimum. A surplus item has quantity greater than twice the minimum. Other items are classified as OK (Table 3.2).

The risk score combines stock coverage and shortage conditions. Coverage is based on the relationship between quantity and demand. Low coverage indicates that the product can become unavailable sooner, while severe shortage conditions increase priority. The model is intentionally rule-based so that the result is explainable to an operator.

Table 3.2 Inventory status classification rules

Condition	Status	Operational Meaning
qty ≤ 0	OUT	Immediate stock-out condition.
qty < min	LOW	Below required minimum.
qty > 2 × min	SURPLUS	Potential source for balancing.
otherwise	OK	Within defined operating range.

3.7 Data Structures Used

The data structures used in the project and the reasons for choosing them are given in Table 3.3.

Table 3.3 Data structures used in the project

Data Structure	Use in Project	Reason
Arrays / primitive arrays	Algorithm buffers, DP tables, graph capacity structures.	Predictable memory and direct indexing.
Trie-like structure	Aho-Corasick and string processing.	Efficient prefix and multi-pattern organization.
Failure links	Aho-Corasick.	Avoid repeated scanning after mismatches.
Adjacency/ capacity structures	Flow network.	Represent warehouse routes and residual capacities.
DP table	Knapsack.	Store best benefit for budget states.
Sets / coverage representation	Supplier prioritization.	Track uncovered risky products.
Reservoir array	Cycle counting.	Maintain fixed-size random sample.
Suffix array and LCP	String laboratory.	Support suffix ordering and longest-common-prefix analysis.

3.8 Algorithm-to-Feature Mapping

Table 3.4 lists the input and output of each algorithm and the warehouse application it supports.

Table 3.4 Algorithm-to-feature mapping

Algorithm	Input	Output	Warehouse Application

Aho-Corasick	Search patterns + product text	Pattern occurrences	Multi-term product search.
KMP	Pattern + text	Match positions	Phrase/string matching.
Z-function	String	Z array	String analysis and large-file experiments.
Rabin-Karp	Pattern + text	Hash-based matches	Alternative streaming search.
Damerau distance	Two strings	Edit distance	Typo correction.
0/1 Knapsack	Candidate costs/ benefits + budget	Selected candidates	Replenishment.
Edmonds-Karp	Capacity graph	Maximum feasible flow	Warehouse transfers.
Greedy Set Cover	Suppliers and covered risky products	Supplier priority list	Supplier prioritization.
Reservoir Sampling	Record stream + k	Uniform sample	Cycle counting.
Quickselect	Numeric values	Order statistic	Percentile analysis.

3.9 Complexity Analysis

The typical time complexities of the techniques used are given in Table 3.5. In the table, n and m denote text and pattern lengths, B is the budget, V and E are the numbers of vertices and edges, and N is the number of records.

Table 3.5 Complexity of the algorithms used

Technique	Typical Complexity	Main Factor
KMP	O(n + m)	Text and pattern length.
Z-function	O(n)	String length.
Rabin-Karp	Expected O(n + m)	Rolling hash and verification.

Aho-Corasick	O(text + matches) after automaton construction	Text length and number of matches.
Damerau edit distance	O(mn) for the DP formulation	Two string lengths.
0/1 Knapsack DP	O(nB)	Candidates n and budget B.
Edmonds-Karp	$O(VE^2)$	Vertices and edges.
Greedy Set Cover	Depends on coverage representation	Number of suppliers/ products.
Reservoir Sampling	O(N)	Stream length.
Quickselect	Expected O(N)	Number of values.

3.10 Datasets Used

Three CSV datasets are used by the application: the inventory dataset (id, sku, name, category, warehouse, supplier, qty, min, price, demand), the route dataset (from, to, capacity_units), and the users dataset (administrator credentials). The inventory covers five warehouse locations: Hyderabad, Mumbai, Delhi, Chennai, and Kolkata. For scalability experiments, a synthetic inventory file of up to 10 million rows (approximately 950 MB) is generated by the system itself. A generated dataset is suitable here because the experiments measure algorithm and I/O behavior, which depends on the size and structure of the data and not on real commercial values.

3.11 Tools and Technologies

Table 3.6 lists the tools and technologies used and the reason each was selected.

Table 3.6 Tools and technologies used

Tool / Technology	Role in the Project	Reason for Selection
Java (JDK)	Algorithm engine, business logic, and server.	Strongly typed, portable, and has built-in support for threads and I/O needed by

Built-in Java HTTP server	Exposes the API endpoints to the browser.	parallel processing.
HTML, CSS, JavaScript	Browser dashboard.	Runs in any modern browser without installation.
CSV files	Persistence for inventory, routes, users, and results.	Simple, transparent, and no database server is required.
Terminal (console) interface	Alternative text-based access.	Uses the same logic and the same data as the browser.
run.sh / run.bat	Compile and launch the application.	One-step start-up on Linux, macOS, and Windows.
Automated self-tests	Validate algorithms and persistence.	Compare results with reference or brute-force behavior.

3.12 Suitability of the Chosen Approach

Each algorithm was selected because its structure matches the structure of the warehouse problem it solves. Multi-term search needs many patterns to be matched in one pass, which is exactly what Aho‑Corasick provides. Replenishment under a fixed budget is a selection problem with additive cost and benefit, which is the 0/1 knapsack problem. Stock transfer under route capacities is a flow problem, so maximum flow guarantees that capacity limits are never exceeded. Supplier prioritization needs a good, explainable cover of risky products rather than a provably minimum one, so greedy set cover is sufficient. Cycle counting needs an unbiased sample from data whose size need not be known in advance, which is the purpose of reservoir sampling. Finally, streaming and parallel processing are chosen for large files because they bound memory use and use the available CPU cores.

4. IMPLEMENTATION

4.1 Implementation Overview

The implementation is centered in the Java application and is supported by a browser page and CSV data files. The application exposes API endpoints through the built-in Java HTTP server, while the terminal interface provides direct access to the same operational logic. The design avoids dependence on an external database or web framework.

The implementation separates routine data handling from algorithmic processing. Inventory records are loaded and represented in memory for normal operations. Large-data functions use streaming or parallel byte ranges when the dataset is too large for convenient full loading. This separation allows the same project to demonstrate both ordinary application development and large-scale algorithmic processing. The principal algorithms are explained below using pseudocode.

4.2 Authentication and Access Control

Authentication begins when a user supplies credentials. The application locates the corresponding user record, computes the password hash using the stored salt, compares it with the stored hash, and creates a random session token after successful validation. Access is limited to the Admin user, so every protected operation first checks that a valid admin session exists.

The important design principle is that access control is enforced by backend logic. The browser may hide or disable controls, but the server-side handler still checks the admin session token before performing the operation.

Pseudocode 4.1: Login and session creation

```
LOGIN(username, password)
    user ← findUser(username)
    IF user does not exist THEN
        RETURN authentication failure
    END IF
    computedHash ← hash(password, user.salt)
    IF computedHash ≠ user.hash THEN
        RETURN authentication failure
```

```
    END IF
    token ← generateSecureRandomToken()
    sessions[token] ← user
    RETURN token
END LOGIN
```

4.3 Inventory and CSV Processing

Inventory records are read from CSV storage and parsed into the internal representation used by the application. Product operations update the shared inventory file so that changes are visible to both browser and terminal interfaces. The project uses temporary-file replacement to reduce the risk of leaving a partially written inventory file if an update is interrupted.

The dashboard can aggregate the loaded inventory by warehouse and category. These summaries are used to report product counts, total units, inventory value, and attention conditions such as low stock, out of stock, and surplus.

Pseudocode 4.2: Inventory update

```
UPDATE_PRODUCT(productId, newValues)
    inventory ← loadInventory()
    product ← findById(inventory, productId)
    IF product does not exist THEN
        RETURN error
    END IF
    validate(newValues)
    replaceFields(product, newValues)
    write inventory to temporary CSV
    atomically replace original CSV
    refresh in-memory view
    RETURN success
END UPDATE_PRODUCT
```

4.4 Intelligent Search

Search accepts one or more query terms and compares them with the

product corpus. Aho-Corasick [2] organizes the search patterns in a trie with failure links, allowing multiple terms to be processed in a single scan. Exact token matches receive stronger ranking than corrected matches. Exact SKU matches receive an additional ranking advantage because SKU is a precise product identifier.

When a token is not found, the vocabulary can be compared using Damerau edit distance [3]. This supports common typing errors, including adjacent transpositions. The search result therefore combines exact retrieval and tolerant correction without replacing precise matches with approximate ones. The search flow is shown in Fig. 4.1 and the corresponding Java source in Fig. 4.2.

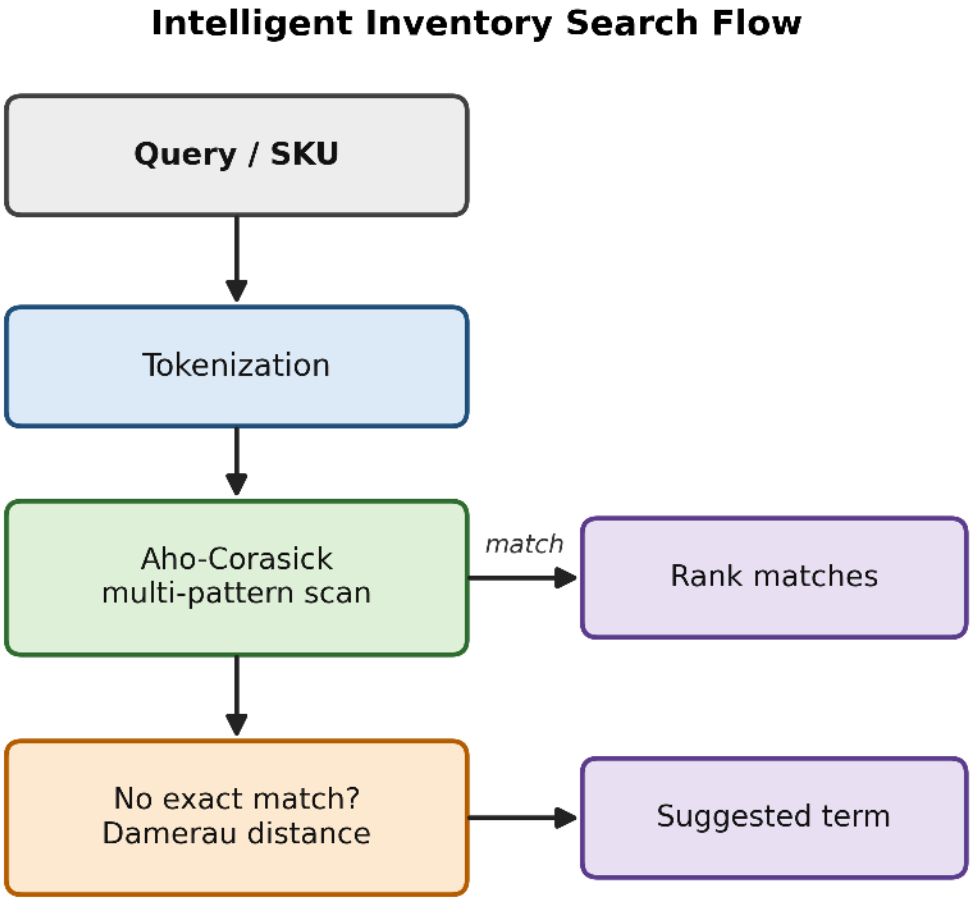


Fig. 4.1 Intelligent search flow combining tokenization, Aho-Corasick matching, ranking, and Damerau-distance typo correction

Pseudocode 4.3: Intelligent inventory search

```
SEARCH(query, inventory)
    tokens ← tokenize(query)
    patterns ← buildSearchPatterns(tokens)
    automaton ← buildAhoCorasick(patterns)
    matches ← scanInventory(automaton, inventory)
    FOR each product in matches
        score ← exactTokenBonus(product, tokens)
        score ← score + phraseBonus(product, query)
        score ← score + exactSKUBonus(product, query)
        rank product using score
    END FOR
    FOR each token not matched
        suggestion ← nearestVocabularyWord(token, DamerauDistance)
        IF suggestion is sufficiently close THEN
            attach suggestion to result
        END IF
    END FOR
    RETURN ranked results
END SEARCH
```

```
// ---------------- Modules 1 + 2: intelligent search ----------------
static final class Idx { long ver; String corpus; int[] off; String[] vocab; int[] freq; String[] low; }
static Idx idx;

static String[] tokenize(String s) {
    String low = s.toLowerCase(Locale.ROOT); ArrayList<String> out = new ArrayList<>(); int i = 0;
    while (i < low.length()) {
        while (i < low.length() && !Character.isLetterOrDigit(low.charAt(i))) i++;
        int j = i; while (j < low.length() && Character.isLetterOrDigit(low.charAt(j))) j++;
        if (j > i) out.add(low.substring(i, j)); i = j;
    }
    return out.toArray(new String[0]);
}

static Idx index() {
    synchronized (Store.LOCK) {
```

Fig. 4.2 Java source implementation of the intelligent search index and tokenization logic

4.5 String Laboratory

The String Laboratory provides an experimental environment for comparing several string algorithms. KMP [1] constructs an LPS-style prefix table to avoid unnecessary comparisons. The Z-function [12] computes prefix matches for every position. Rabin-Karp [11] uses a rolling hash to compare windows efficiently. Aho-Corasick processes multiple patterns through a trie and failure links. The suffix-array and LCP functionality [13] provides a different view of the same text by ordering suffixes and measuring common prefixes.

The laboratory is valuable academically because it demonstrates that the same search problem can be approached with different structures and complexity trade-offs. It also allows the large-data experiments to use streaming versions of string matching instead of loading an entire file into memory.

Pseudocode 4.4: KMP search

```
KMP_SEARCH(text, pattern)
    lps ← buildLPS(pattern)
    i ← 0
    j ← 0
    WHILE i < length(text)
        IF text[i] = pattern[j] THEN
            i ← i + 1
            j ← j + 1
            IF j = length(pattern) THEN
                report match at i - j
                j ← lps[j - 1]
            END IF
        ELSE IF j > 0 THEN
            j ← lps[j - 1]
        ELSE
            i ← i + 1
        END IF
    END WHILE
END KMP_SEARCH
```

Pseudocode 4.5: Rabin-Karp streaming search

```
RABIN_KARP(textStream, pattern)
    patternHash ← hash(pattern)
    windowHash ← hash(first window)
    FOR each next character/window
        IF windowHash = patternHash THEN
            verify characters
            IF equal THEN
                report match
            END IF
        END IF
        update rolling hash
    END FOR
END RABIN_KARP
```

The Java implementation of the algorithm engine, including the pseudo-random generator and the KMP matcher, is shown in Fig. 4.3.

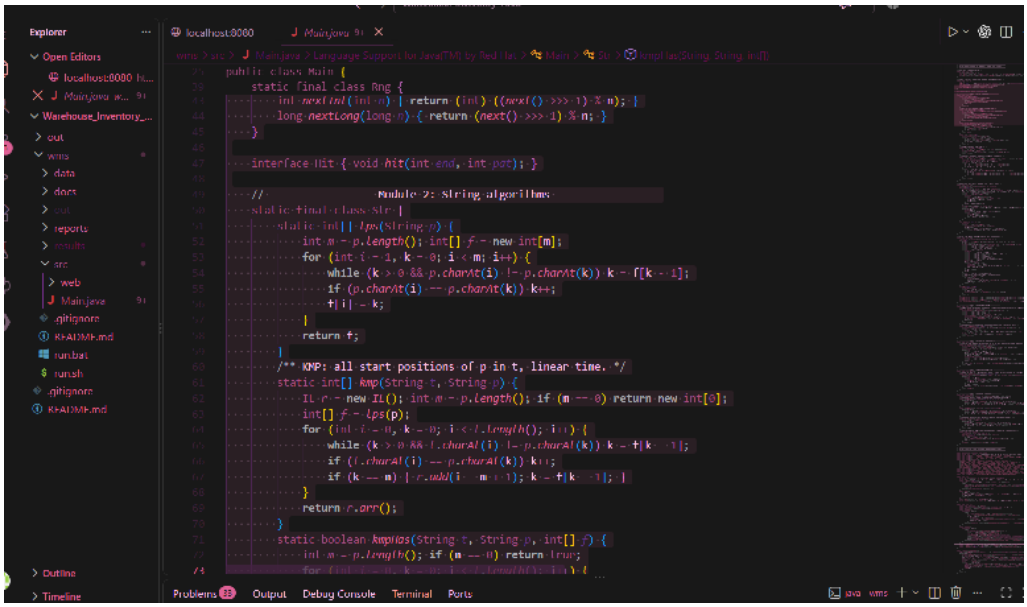


Fig. 4.3 Java source implementation showing the hand-built algorithm engine, pseudo-random generator, and KMP string-matching logic

4.6 Replenishment

The replenishment module begins with low-stock candidates. For each candidate, the project estimates an order quantity that moves stock

toward twice the minimum level. Cost is estimated from order quantity and unit price. A benefit score reflects the operational importance of restoring stock, with shortage conditions receiving higher priority.

The candidate set is then solved as a 0/1 knapsack problem [4]. Each candidate is treated as an item that can either be selected or not selected. The budget is the capacity. The dynamic-programming table records the best benefit achievable for each budget level. A reconstruction step identifies the selected candidates. The complete flow is shown in Fig. 4.4.

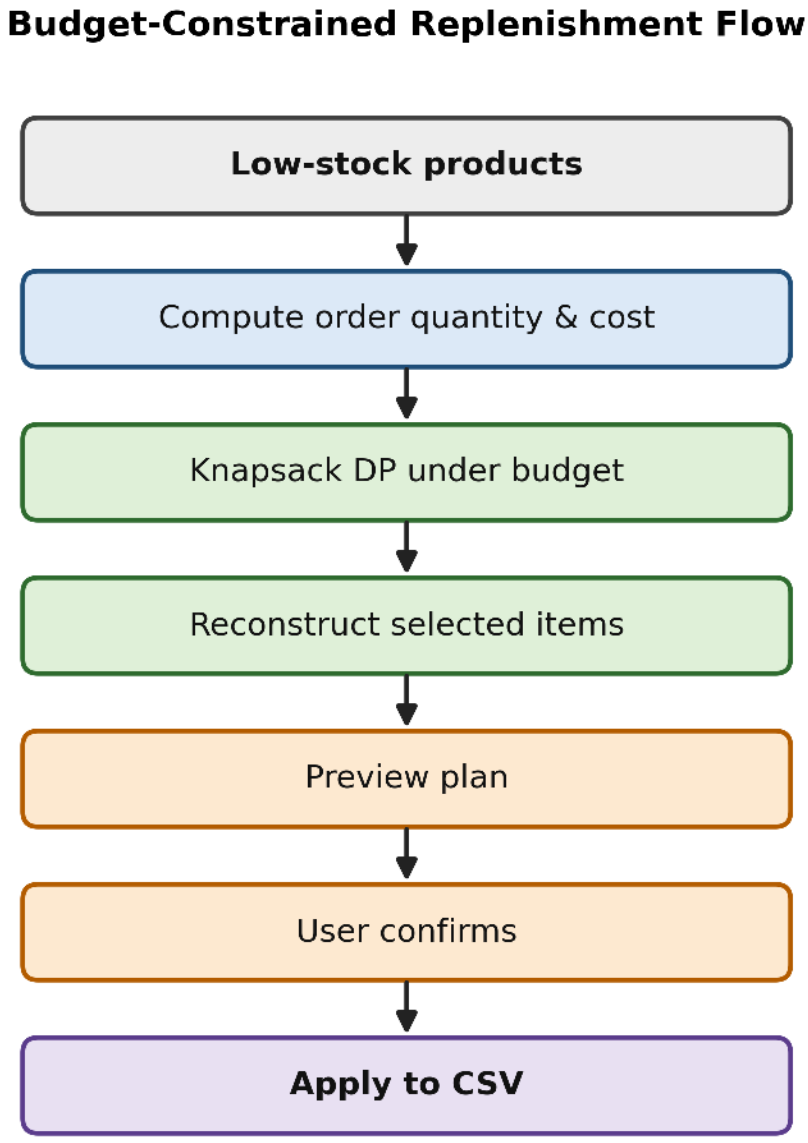


Fig. 4.4 Replenishment flow from low-stock detection to knapsack optimization, preview, confirmation, and CSV update

Pseudocode 4.6: Budget-constrained replenishment

```
REPLENISH(inventory, budget)
    candidates ← all products with qty < min
    FOR each candidate
        orderQty ← 2 × min - qty
        cost ← orderQty × price
        benefit ← businessPriority(orderQty, demand, qty)
    END FOR
    dp[0..budget] ← 0
    choose[0..budget] ← empty
    FOR each candidate i
        FOR b from budget DOWN TO cost[i]
            IF dp[b - cost[i]] + benefit[i] > dp[b] THEN
                dp[b] ← dp[b - cost[i]] + benefit[i]
                choose[b] ← i
            END IF
        END FOR
    END FOR
    selected ← reconstructSelectedItems(choose, dp)
    RETURN selected
END REPLENISH
```

4.7 Warehouse Balancing

Warehouse balancing identifies surplus and shortage warehouses for a SKU. Surplus warehouses are connected from a source node according to their available surplus. Shortage warehouses connect to the sink according to their shortage. Warehouse-to-warehouse routes become directed edges with explicit capacity.

Edmonds-Karp [5] repeatedly performs breadth-first search to find an augmenting path in the residual network. The minimum residual capacity on the path is sent as additional flow, after which forward and reverse residual capacities are updated. When no augmenting path remains, the positive flows can be translated into a transfer plan. Unmet shortage can be reported rather than assuming unlimited transportation capacity.

Pseudocode 4.7: Warehouse balancing with maximum flow

```
BALANCE(sku)
    source ← createSource()
    sink ← createSink()
    FOR each warehouse
        IF stock(warehouse, sku) > 2 × min(warehouse, sku) THEN
            addEdge(source, warehouse, surplusAmount)
        ELSE IF stock(warehouse, sku) < min(warehouse, sku) THEN
            addEdge(warehouse, sink, shortageAmount)
        END IF
    END FOR
    FOR each route
        addEdge(route.from, route.to, route.capacity)
    END FOR
    flow ← 0
    WHILE BFS finds augmenting path source → sink
        pathFlow ← minimum residual capacity on path
        update residual capacities
        flow ← flow + pathFlow
    END WHILE
    RETURN transfer plan derived from positive flows
END BALANCE
```

4.8 Risk and Supplier Prioritization

Risk analysis first classifies inventory according to stock conditions and coverage. Products with severe shortage or low coverage receive greater operational attention. The supplier-prioritization stage then builds a coverage set for each supplier containing risky products that the supplier can address.

The greedy set-cover approximation [6] repeatedly chooses the supplier that covers the largest number of currently uncovered risky products. This produces a practical priority order, but it does not guarantee the globally minimum supplier set in every case.

Pseudocode 4.8: Greedy supplier prioritization

```
SUPPLIER_PRIORITY(atRiskProducts, suppliers)
    uncovered ← set(atRiskProducts)
    selected ← empty list
```

```
    WHILE uncovered is not empty
        bestSupplier ← supplier covering the largest number of
        uncovered products
        IF no supplier adds new coverage THEN
            BREAK
        END IF
        add bestSupplier to selected
        remove its covered products from uncovered
    END WHILE
    RETURN selected
END SUPPLIER_PRIORITY
```

4.9 Cycle Counting

Cycle counting is an audit technique in which a subset of inventory records is physically checked instead of requiring every record to be counted at once. The project uses reservoir sampling [7] so that a fixed-size sample can be selected from a stream without knowing the complete stream size in advance.

After a sample is generated, the operator can enter counted quantities. The system can record discrepancies and, where supported by the workflow, apply the counted quantity as an inventory adjustment.

Pseudocode 4.9: Reservoir sampling

```
RESERVOIR(stream, k)
    reservoir ← first k items from stream
    FOR i from k + 1 to N
        j ← random integer in [1, i]
        IF j ≤ k THEN
            reservoir[j] ← stream[i]
        END IF
    END FOR
    RETURN reservoir
END RESERVOIR
```

4.10 Large Data Processing

The large-data module is designed for datasets of up to 10 million rows in the documented experiments. The key implementation principle is to avoid loading the complete file into memory. For aggregation, the file can be divided into byte ranges and processed by multiple workers. Each worker advances to a complete line boundary before processing records, preventing a record from being split between workers.

Streaming versions of KMP, Z-function, and Rabin-Karp scan large files without materializing the complete text. Reservoir sampling maintains only a fixed-size sample. Quickselect [8] computes order statistics without requiring a full sort of every value.

Pseudocode 4.10: Parallel large-file aggregation

```
PARALLEL_AGGREGATE(file, workers)
    ranges ← splitFileIntoByteRanges(file, workers)
    results ← empty collection
    PARALLEL FOR each range
        move to next complete line boundary
        localResult ← scanAndAggregate(range)
        append localResult to results
    END PARALLEL FOR
    RETURN reduce(results)
END PARALLEL_AGGREGATE
```

Pseudocode 4.11: Quickselect percentile

```
QUICKSELECT(values, k)
    choose pivot
    partition values around pivot
    IF pivot position = k THEN
        RETURN pivot
    ELSE IF k < pivot position THEN
        recurse on left partition
    ELSE
        recurse on right partition
    END IF
END QUICKSELECT
```

4.11 API Integration

The browser communicates with the Java server through HTTP endpoints. The endpoint groups and their purposes are listed in Table 4.1.

Table 4.1 API endpoint groups

Endpoint Group	Purpose
/api/login, /api/logout	Authentication and session control.
/api/meta	Metadata such as categories and warehouses.
/api/dashboard	KPIs, warehouse summaries, category summaries, attention list.
/api/products/*	Inventory CRUD operations.
/api/replenish and /api/replenish/apply	Preview and apply replenishment.
/api/balance and /api/balance/apply	Preview and apply warehouse transfers.
/api/risk	Risk ranking and supplier priority information.
/api/cycle and /api/cycle/record	Sampling and cycle-count recording.
/api/strings	String-algorithm laboratory.
/api/large/*	Large-data generation and analysis.

4.12 Front-End Workflow

The browser dashboard presents user-oriented operations instead of exposing internal algorithm names as the primary interface. A user can search for inventory, review attention items, generate a replenishment plan, inspect a balancing plan, review risk and supplier priority information, perform cycle counting, and access large-data operations. The JavaScript layer sends API requests and renders returned data.

The front end therefore acts as an interaction layer rather than duplicating the algorithm engine. This separation allows the same underlying logic to

be accessed through the terminal interface.

Pseudocode 4.12: Generic browser operation

```
USER_ACTION(action)
    validate required input
    token ← current session token
    response ← HTTP request(action, input, token)
    IF response indicates authentication failure THEN
        show login prompt
    ELSE IF response contains error THEN
        show validation/error message
    ELSE
        render returned result
    END IF
END USER_ACTION
```

4.13 Summary of Implementation

The implementation maps every warehouse operation to one algorithm and exposes it through a common API used by both the browser and the terminal. The pseudocode in this chapter describes the logic of each module; the tests and measurements that validate it are presented in Chapter 5.

5. RESULTS

5.1 Test Environment

The project test evidence includes automated algorithm and integration tests and a separate performance report. The large-data performance measurements were recorded on a 1-core cloud virtual machine for a dataset of approximately 950 MB. These measurements are project-reported values and depend on the test environment; they should not be interpreted as universal hardware-independent benchmarks. The test areas and results are summarized in Table 5.1.

Table 5.1 Summary of test areas and reported results

Test Area	Validation	Reported

		Result
String matching	KMP, Z-function, Rabin-Karp, suffix-array behavior compared with reference behavior	PASS
Aho-Corasick	Multi-pattern counts compared with naive matching	PASS
Edit distance	Damerau transposition and typo-correction behavior	PASS
Knapsack	DP result compared with brute-force optimum on test cases	PASS
Maximum flow	Classical network and max-flow/min-cut consistency	PASS
Set cover	Greedy supplier coverage behavior	PASS
Randomized algorithms	Quickselect/quicksort and reservoir checks	PASS
Persistence	CSV round-trip and inventory updates	PASS
Application search	Typo correction and exact-SKU ranking	PASS
Optimization	Budget constraint and warehouse balancing	PASS
Large data	Parallel vs sequential aggregation and large-file search	PASS
Percentiles	Quickselect-based percentile calculations	PASS

5.2 Functional Test Strategy

The testing strategy combines unit-style algorithm validation with application-level integration checks. The algorithm tests are important because optimization and graph algorithms can appear correct on small examples while failing on boundary conditions. Reference comparisons and brute-force checks provide a stronger basis for validation.

Application-level checks confirm that the algorithmic output can be connected to inventory state and persisted results. The large-data tests additionally compare parallel and sequential aggregation to verify that

concurrency does not alter the final result.

5.3 Individual Test Results

5.3.1 Exact and SKU Search

Exact product/SKU queries were checked to ensure that precise identifiers receive appropriate ranking. Reported result: PASS.

5.3.2 Multi-Term Search

Multiple search terms were processed through the string-search workflow. Reported result: PASS.

5.3.3 Typo Correction

Misspelled terms including transposition-style errors were evaluated using Damerau distance. Reported result: PASS.

5.3.4 KMP

KMP results were compared against reference matchingbehavior. Reported result: PASS.

5.3.5 Rabin-Karp

Rolling-hash search was validated against reference results. Reported result: PASS.

5.3.6 Z-function

Z-array-based analysis was checked against expected stringbehavior. Reported result: PASS.

5.3.7 Aho- Corasick

Multi-pattern counts were compared with naive matching. Reported result: PASS.

5.3.8 Knapsack

Dynamic-programming results were compared with brute-force optimum for test cases. Reported result: PASS.

5.3.9 Maximum Flow

Flow results were checked for consistency with classical network properties. Reported result: PASS.

5.3.10 Greedy Set Cover

Supplier coverage behavior was validated. Reported result: PASS.

5.3.11 Reservoir Sampling

Sampling logic was tested for expected fixed-size behavior. Reported result: PASS.

5.3.12 CSV Persistence

Inventory updates and round-trip persistence were tested. Reported result: PASS.

5.3.13 Large-Data Aggregation

Parallel and sequential aggregation were compared. Reported result: PASS.

5.3.14 Percentile Calculation

Quickselect-based percentile values were checked. Reported result: PASS.

5.4 Performance Results

The reported execution times of selected large-dataset operations are given in Table 5.2 and plotted in Fig. 5.1.

Table 5.2 Reported performance of large-dataset operations

Operation	Reported Time	Dataset / Context
Generate 10 million rows	~8.0 seconds	Large synthetic inventory dataset.
Parallel aggregation	~5.7 seconds	Large dataset, 1-core cloud VM report.
KMP search	~3.1 seconds	Large-file string search.
Rabin-Karp search	~6.3 seconds	Large-file string search.
Quickselect percentiles	~2.6 seconds	Large numeric analysis.
Reservoir sampling	~1.2 seconds	Large record stream.

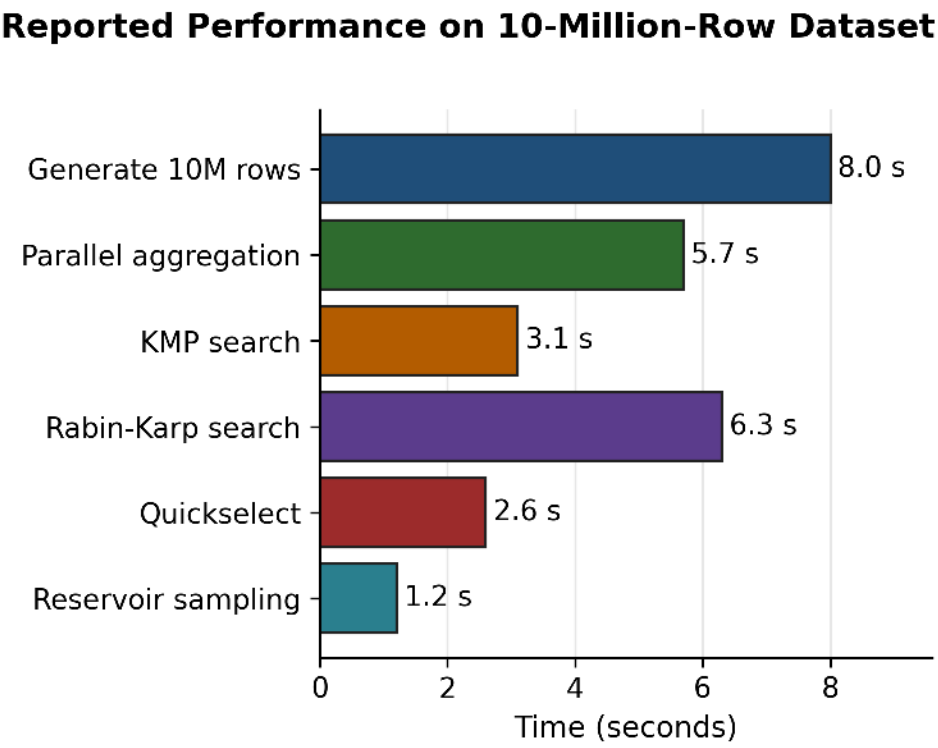


Fig. 5.1 Reported execution times for selected large-dataset operations

5.5 Dashboard Results

The dashboard consolidates operational indicators into a single view. The project reports product counts, total units, inventory value, low-stock and out-of-stock conditions, surplus items, warehouse summaries, category summaries, and an attention list. The purpose of the dashboard is to move the user from raw records to operational decisions (Fig. 5.2 and Fig. 5.3).

A dashboard does not replace the algorithms. Instead, it provides the context required to decide which algorithmic workflow should be executed next. For example, a high-risk attention list can lead to replenishment analysis, balancing analysis, or supplier prioritization.

5.6 Replenishment Results

The replenishment module produces a candidate plan instead of automatically ordering every low-stock product. The plan respects the supplied budget through the knapsack constraint. The preview/apply

workflow allows an operator to inspect the candidate set before inventory changes are applied (Fig. 5.4).

The result format includes product identity, quantity, estimated cost, and priority/benefit information. This makes the output more understandable than a raw DP table.

5.7 Warehouse Balancing Results

The balancing module transforms surplus and shortage information into feasible transfer suggestions. Because route capacities are represented explicitly in the flow network, a generated transfer cannot exceed the capacity of a configured route. If total shortage cannot be satisfied, the flow model can report the remaining unmet shortage instead of assuming unlimited movement (Fig. 5.5).

5.8 Search Results

The search implementation provides exact and tolerant behavior. Exact SKU matching receives a ranking advantage. Multiple tokens can be processed using Aho-Corasick, while Damerau edit distance supplies suggestions for terms that are not found directly. The supplied tests validate typo correction, including transposition-style errors.

This combination is useful in a warehouse environment because operators often search by product name or SKU under time pressure. The system therefore aims to preserve exact-match precision while improving tolerance to imperfect input.

5.9 Large-Data Results

The large-data experiments demonstrate that the project can operate on data volumes substantially larger than the normal interactive inventory. Streaming search avoids materializing the entire file. Reservoir sampling maintains a fixed-size audit sample. Quickselect computes percentiles without a full sort. Parallel aggregation divides work across ranges and combines partial results (Fig. 5.8).

The reported measurements show that these techniques are practical for the project's test environment. They also demonstrate the difference between algorithmic complexity and application-level performance: a theoretically efficient algorithm still depends on file I/O, parsing, concurrency overhead, and the hardware environment.

5.10 Complexity and Interpretation

The functional results indicate that the project successfully connects theoretical algorithms to practical inventory tasks. The 23-pass, zero-failure automated test report is evidence of the submitted implementation's tested behavior. The large-data measurements provide an additional performance perspective.

The most important result is not a single runtime number. It is the successful mapping of different algorithmic paradigms to different warehouse problems. String algorithms solve search, dynamic programming solves budget allocation, network flow solves constrained movement, greedy approximation supports supplier coverage, and randomized/streaming methods support scalable auditing and analytics.

5.11 Project Screenshots

The following output screenshots demonstrate the major browser-based and terminal workflows implemented in the Warehouse Inventory Management and Optimization System.

Fig. 5.2 shows the system overview dashboard.

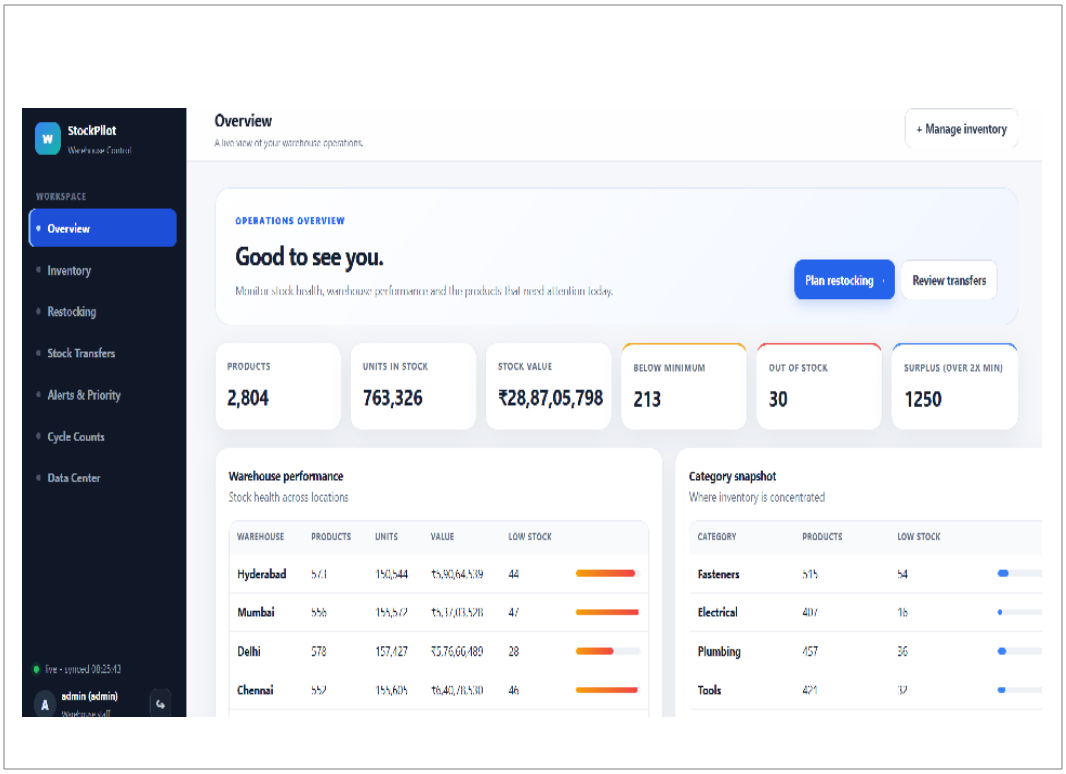


Fig. 5.2 System overview dashboard showing warehouse-wide operational status

Fig. 5.3 shows the inventory management interface.

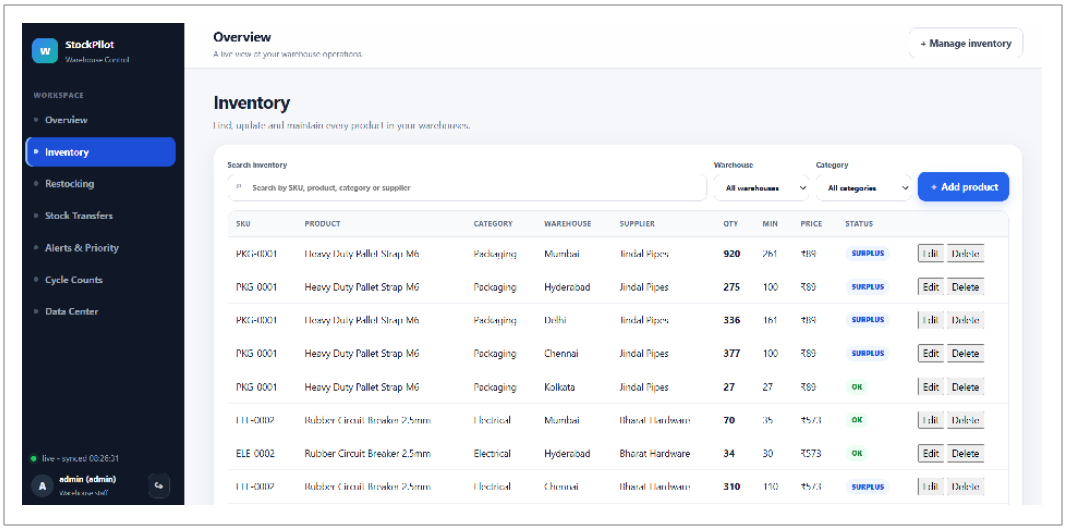


Fig. 5.3 Inventory management interface showing product records and stock status

Fig. 5.4 shows the restocking planner.

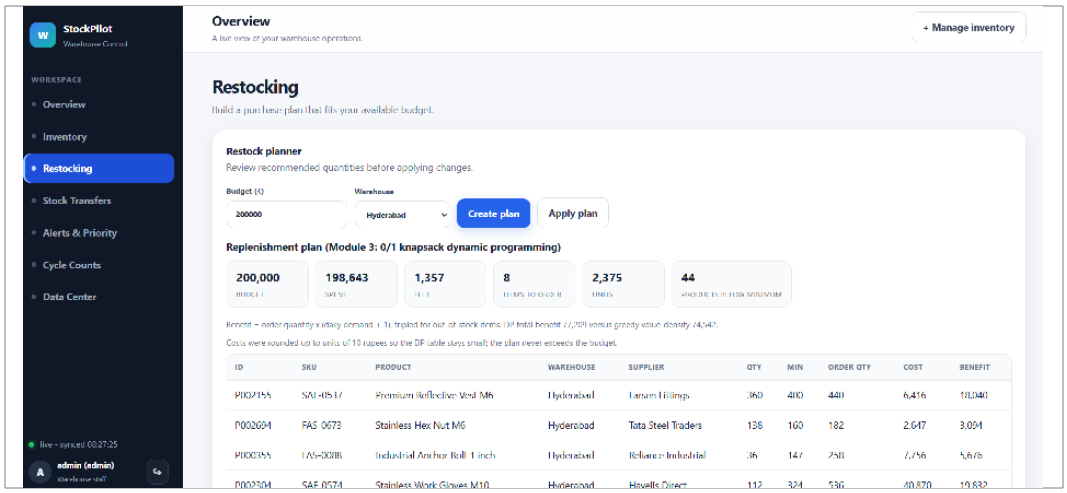


Fig. 5.4 Restocking planner showing the budget-constrained replenishment plan

Fig. 5.5 shows the warehouse transfer planner.

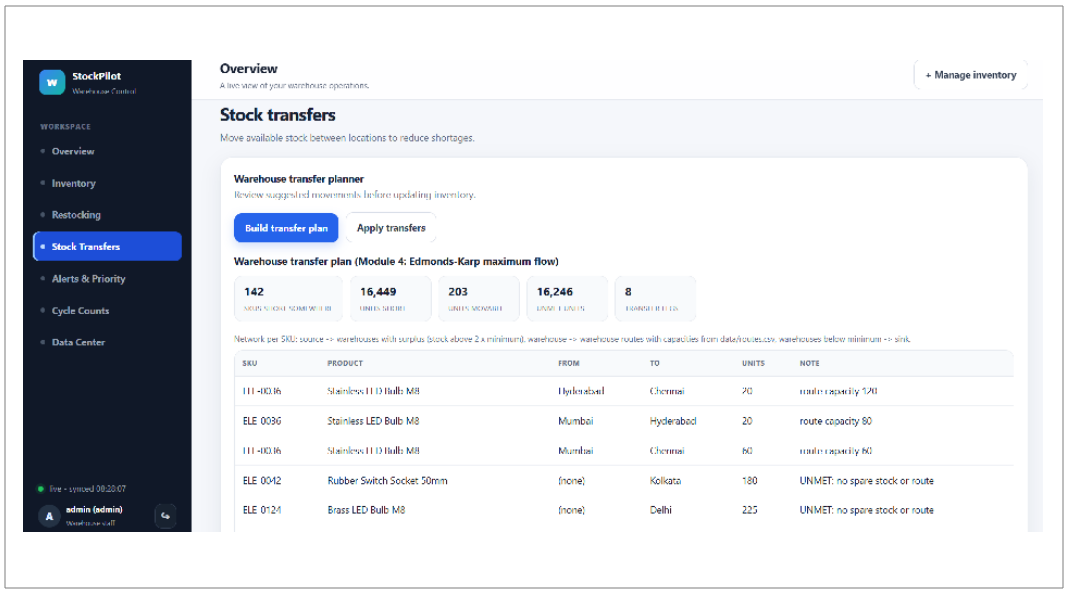


Fig. 5.5 Warehouse transfer planner showing maximum-flow based stock movements

Fig. 5.6 shows the alerts and priority view.

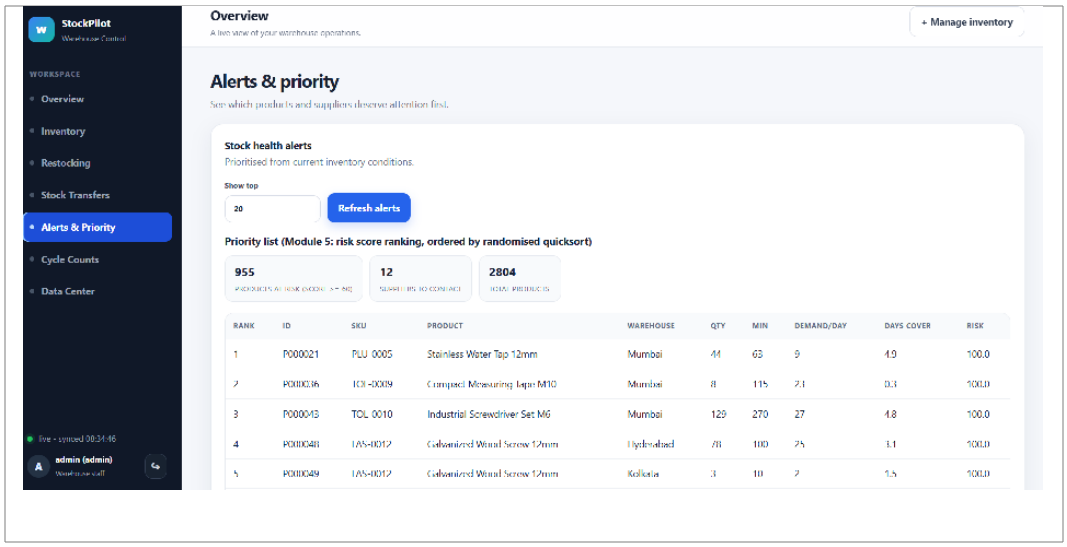


Fig. 5.6 Alerts and priority view showing risk-ranked products

Fig. 5.7 shows the cycle-count planner.

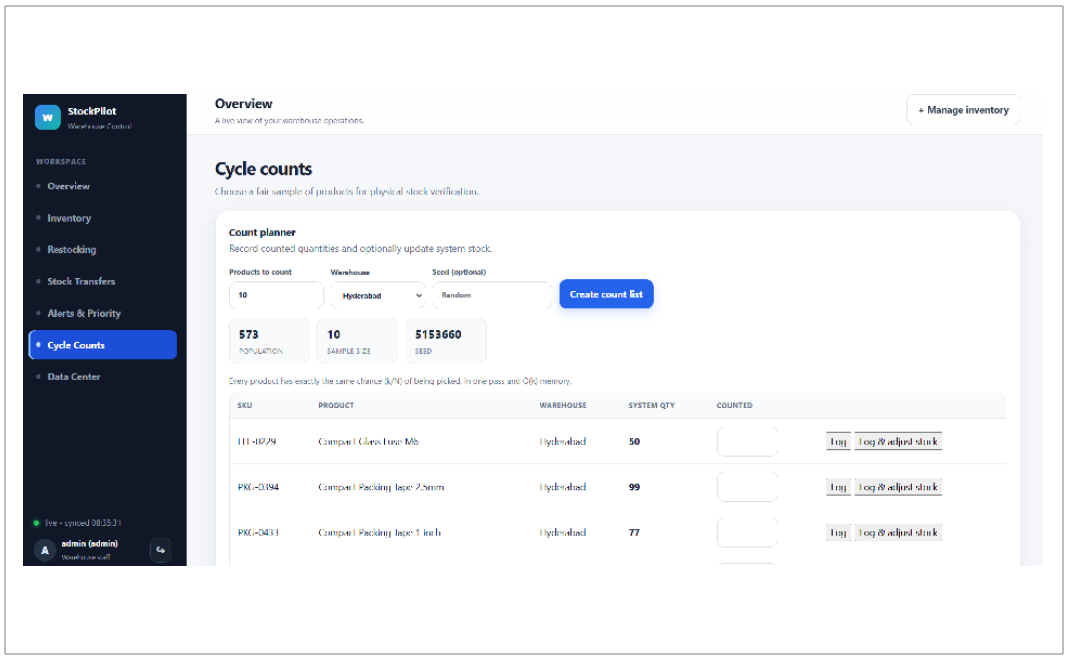


Fig. 5.7 Cycle-count planner showing reservoir-sampled products for physical verification

Fig. 5.8 shows the data center interface.

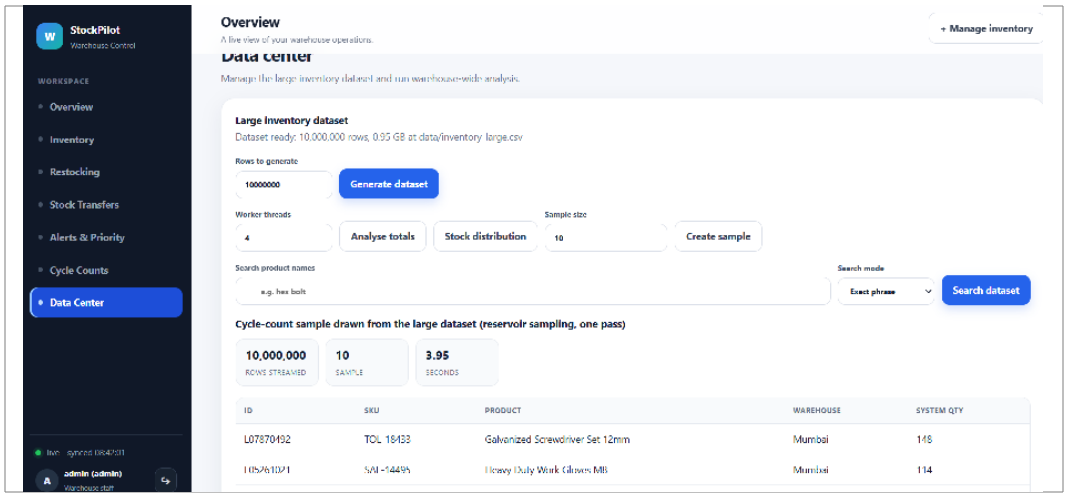


Fig. 5.8 Data center interface for large-dataset generation, analysis, sampling, and search

Fig. 5.9 shows the login interface.

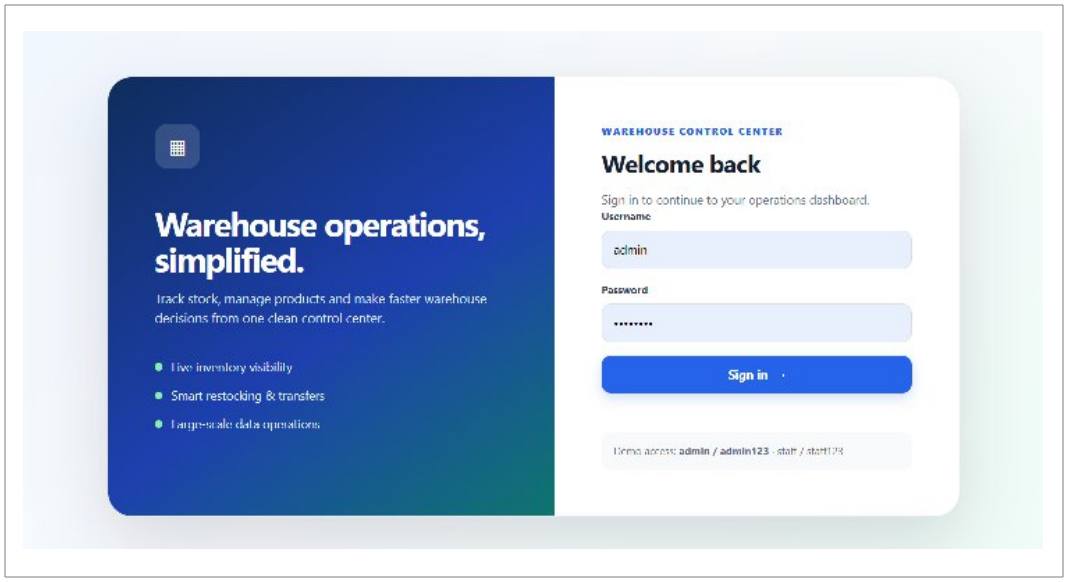


Fig. 5.9 Warehouse Control Center login interface used for authenticated access

Fig. 5.10 shows the terminal interface.

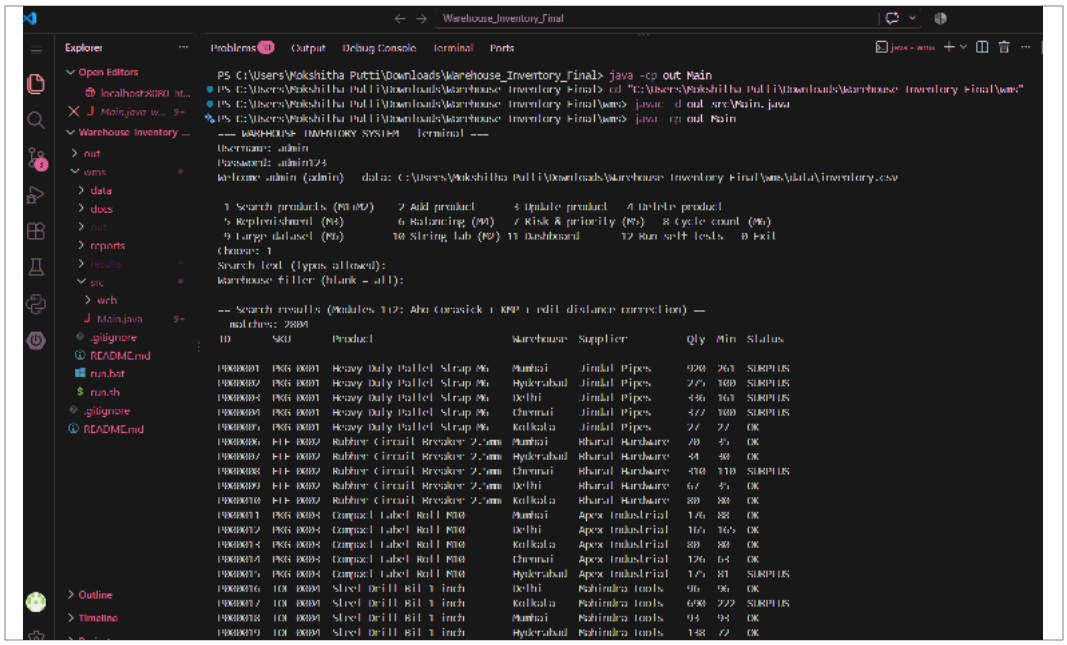


Fig. 5.10 Terminal interface showing the authenticated warehouse system and available inventory-management operations

6. CHALLENGES AND LIMITATIONS

6.1 Issues Faced During Development

6.1.1 Translating Business Rules

Warehouse concepts such as shortage, surplus, risk, and demand must be converted into precise computational rules. Ambiguous definitions can cause inconsistent results across modules.

6.1.2 Integrating Multiple Algorithms

The project uses several unrelated algorithm families. Their interfaces, input formats, and output structures must remain consistent.

6.1.3 Search Ranking

Exact matches, phrase matches, multi-term matches, and typo suggestions need to be combined without allowing approximate matches to obscure precise identifiers.

6.1.4 Optimization Modeling

Replenishment must convert a practical purchasing decision into cost/ benefit values suitable for knapsack optimization.

6.1.5 Flow Network Construction

Warehouse balancing requires correct source/sink construction, route direction, residual capacities, and translation of positive flow into transfers.

6.1.6 Large Files

Large datasets cannot be treated like small CSV files. Full loading creates memory pressure, so streaming and range processing are required.

6.1.7 Shared Persistence

The browser and terminal interfaces use the same inventory file, making safe update and refresh behavior important.

6.1.8 API/UI Integration

Algorithmic results must be converted into responses that are understandable and usable by the browser interface.

6.2 Solutions Applied

6.2.1 Centralized Inventory Rules

Status and risk calculations are kept in common inventory-analysis logic so that the browser and terminal do not implement different definitions.

6.2.2 Preview/Apply Workflow

Replenishment and balancing expose a preview stage before applying changes, reducing the risk of accidental updates.

6.2.3 Reference-Based Testing

Algorithm outputs are compared with reference or brute-force behavior where appropriate.

6.2.4 Safe File Replacement

Inventory updates are written through temporary-file replacement to reduce partial-write risk.

6.2.5 Streaming and Range Processing

Large files are processed in streams or byte ranges rather than being fully materialized.

6.2.6 Parallel Validation

Parallel aggregation is compared with sequential aggregation to verify correctness.

6.2.7 Backend Access Control

Admin-session checks are enforced in backend operations instead of relying only on interface visibility.

6.3 Limitations of the System

6.3.1 CSV-Based Persistence

CSV storage is simple and transparent but does not provide the transactions, indexing, concurrency controls, backup, and recovery mechanisms of a production database.

6.3.2 Rule-Based Risk Model

The risk model uses defined stock and coverage rules and does not learn seasonality, lead times, supplier reliability, or probabilistic stock-out behavior.

6.3.3 Simplified Replenishment Objective

The current replenishment model does not include every real-world purchasing constraint such as minimum order quantities, supplier contracts, delivery lead times, or multi-period planning.

6.3.4 Greedy Supplier Approximation

Greedy set cover is an approximation and does not guarantee the globally minimum supplier set in every instance.

6.3.5 Environment-Dependent Performance

The large-data timings depend on the reported hardware and workload and should not be treated as universal benchmarks.

6.3.6 Lightweight Web Interface

The browser interface is an academic prototype and does not include enterprise-scale observability, deployment, audit, or security infrastructure.

6.4 Security and Operational Considerations

The project includes basic password hashing, salted credentials, and session tokens for the admin user. These features demonstrate authentication and authorization concepts but do not make the system a hardened production identity platform.

A production deployment would additionally require password policies, session expiration, secure cookie configuration, transport encryption, rate limiting, audit logs, centralized identity management, backup procedures, and stronger concurrent-update handling. These requirements are outside the scope of the current academic implementation.

6.5 Summary of Challenges and Limitations

The challenges encountered during development are closely related to the project's central objective: integrating multiple DSA techniques into one operational system. The solutions focus on separating concerns, validating algorithms, making optimization changes reviewable, and avoiding unnecessary memory use.

The limitations do not invalidate the project. Instead, they define the boundary between an academic prototype and a production warehouse-management platform. This distinction is important when interpreting the performance results and future-work proposals.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

The Warehouse Inventory Management and Optimization System demonstrates how Data Structures and Algorithms can be integrated into a realistic software application. Rather than treating algorithms as isolated classroom exercises, the project maps them to concrete

warehouse problems.

Aho-Corasick, KMP, Z-function, Rabin-Karp, suffix-array/LCP processing, and Damerau edit distance support intelligent search and string analysis. Dynamic programming supports budget-constrained replenishment. Edmonds-Karp maximum flow supports warehouse balancing under route capacities. Greedy set cover supports supplier prioritization. Reservoir sampling supports cycle counting, while streaming, parallel aggregation, and quickselect support large-data analysis.

The project also demonstrates software-engineering integration. The browser dashboard and terminal interface use common Java logic and shared CSV persistence. Admin authentication protects all operations. Preview/apply workflows provide a human review point before optimization changes are committed.

The supplied test evidence reports 23 passes and 0 failures, and the performance evidence demonstrates operations over datasets of up to 10 million records. The project therefore meets its academic objective of showing how multiple algorithmic approaches can be selected according to the structure of a practical problem and integrated into a functioning application.

7.2 Future Work

7.2.1 Transactional Database Storage

Replace CSV persistence with a transactional relational or distributed database and introduce proper concurrency control.

7.2.2 Demand Forecasting

Use historical demand, seasonality, and trend information instead of relying only on current demand values.

7.2.3 Advanced Replenishment

Add supplier lead time, minimum order quantities, discounts, multi-period planning, and other purchasing constraints.

7.2.4 Transport-Aware Balancing

Extend the flow model to include transport cost, delivery time, vehicle capacity, and route reliability.

7.2.5 Stronger Security

Introduce secure cookies, password policies, rate limiting, audit logging, encrypted transport, and enterprise identity integration.

7.2.6 Multiple User Roles

Add additional user roles, such as warehouse staff with limited permissions, alongside the Admin user, with configurable dashboards for each.

7.2.7 Historical Analytics

Add trend visualizations for inventory value, stock-outs, service levels, and demand.

7.2.8 Larger-Scale Processing

Explore distributed processing or columnar storage for datasets substantially larger than the current experiment.

7.2.9 Automated Alerts

Add scheduled replenishment, risk, and stock-out alerts.

7.2.10 Search Ranking Improvements

Combine SKU precision, phrase relevance, fuzzy similarity, warehouse context, and category relevance into richer ranking.

7.2.11 Better Supplier Optimization

Extend supplier selection with cost, lead time, reliability, and service-level constraints.

7.2.12 Automated Reporting

Generate scheduled operational reports for managers and auditors.

7.2.13 Expanded Testing

Increase automated tests for boundary cases, malformed input, large

workloads, and concurrent requests.

7.3 Skills Learned During Development

- Algorithm selection and complexity analysis.
- Dynamic programming and optimization modeling.
- Graph modeling and maximum-flow implementation.
- String processing with tries, failure links, hashing, and suffix arrays.
- Edit-distance and approximate matching.
- Approximation algorithms and set-cover reasoning.
- Randomized algorithms and reservoir sampling.
- Streaming and parallel file processing.
- Java HTTP server and API design.
- CSV persistence and data transformation.
- Authentication, sessions, and admin access control.
- Automated testing and reference-based validation.
- Performance benchmarking and interpretation.
- Debugging and integration of algorithmic modules.
- Technical documentation and project presentation.

REFERENCES

[1] D. E. Knuth, J. H. Morris, Jr., and V. R. Pratt, “Fast pattern matching in strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.

[2] A. V. Aho and M. J. Corasick, “Efficient string matching: An aid to bibliographic search,” Communications of the ACM, vol. 18, no. 6, pp. 333–340, 1975.

[3] F. J. Damerau, “A technique for computer detection and correction of spelling errors,” Communications of the ACM, vol. 7, no. 3, pp. 171–176, 1964.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction

to Algorithms, 3rd ed., MIT Press, 2009.

[5] J. Edmonds and R. M. Karp, “Theoretical improvements in algorithmic efficiency for network flow problems,” Journal of the ACM, vol. 19, no. 2, pp. 248–264, 1972.

[6] D. S. Johnson, “Approximation algorithms for combinatorial problems,” Journal of Computer and System Sciences, vol. 9, no. 3, pp. 256–278, 1974.

[7] J. S. Vitter, “Random sampling with a reservoir,” ACM Transactions on Mathematical Software, vol. 11, no. 1, pp. 37–57, 1985.

[8] C. A. R. Hoare, “Algorithm 65: Find,” Communications of the ACM, vol. 4, no. 7, p. 321, 1961.

[9] Oracle, Java Platform, Standard Edition API Documentation, used for Java runtime and standard-library concepts implemented in the project.

[10] Project source code and supplied project documentation, Warehouse Inventory Management and Optimization System, including README, design notes, test report, performance report, datasets, and implementation files.

[11] R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987.

[12] D. Gusfield, Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology, Cambridge University Press, 1997.

[13] U. Manber and G. Myers, “Suffix arrays: A new method for on-line string searches,” SIAM Journal on Computing, vol. 22, no. 5, pp. 935–948, 1993.